

Zabbix 통합 모니터링 시스템 구축 보고서

구분	내용
프로젝트명	KINX 통합 모니터링 시스템 전환 프로젝트
주요 범위	Zabbix 7.0 업그레이드, 고가용성 구축, Nagios 통합, 부하 및 장애조치 검증
핵심 기술	Zabbix 7.0 LTS, PostgreSQL 16, TimescaleDB, Patroni, etcd, HAProxy, Zabbix Proxy Group
구축 형태	Zabbix Server·Proxy·Database 계층별 이중화
작성자	조영빈

Executive Summary

본 프로젝트는 기존 **Nagios와 Zabbix 6.0 LTS로 분산되어 있던 모니터링 체계를 Zabbix 7.0 LTS 기반의 단일 통합 모니터링 플랫폼으로 전환**하고, 모니터링 시스템의 성능·확장성·가용성을 강화하기 위해 수행하였다.

기존 환경은 Zabbix Server와 Database가 단일 VM에 집중되어 있어 장애 발생 시 모니터링 기능 전체가 중단될 수 있었으며, Nagios와 Zabbix에서 동일한 서버를 서로 다른 이름으로 관리하는 경우가 있어 감시 대상의 중복과 누락 가능성이 존재했다. 또한 Nagios에서 사용하던 DNS 및 RTMP 스트리밍 검사는 Zabbix 기본 템플릿만으로 직접 대체하기 어려웠다.

이에 따라 다음과 같은 개선을 수행하였다.

- Zabbix 6.0 LTS에서 Zabbix 7.0 LTS로 버전 전환
- MySQL 기반 데이터베이스를 PostgreSQL 16 + TimescaleDB로 전환
- Zabbix Agent에서 Zabbix Agent 2로 업그레이드
- Zabbix Server, Proxy, Database 계층별 이중화
- Patroni, etcd, HAProxy를 이용한 PostgreSQL 자동 장애조치 구성
- Zabbix Proxy Group 기반 수집 계층 로드밸런싱 및 페일오버 구성
- Nagios 259개 호스트와 Zabbix 호스트 간 전수 매핑
- DNS Check 및 RTMP Stream Check 커스텀 템플릿 개발
- 2,000개 Dummy Host와 5,000 NVPS 부하 환경 구현
- Database, Zabbix Server, Zabbix Proxy 장애조치 시나리오 검증

1. 프로젝트 개요

1.1. 추진 배경

기존 모니터링 환경은 Nagios와 Zabbix 6.0 LTS가 병행 운영되는 구조였다. 두 시스템은 일부 동일한 서버와 서비스를 중복으로 감시하고 있었으나, 각 시스템에서 사용하는 Hostname 및 Naming Convention이 서로 달랐다.

예를 들어 Nagios에서는 다음과 같이 호스트를 관리했다.

```
Autocdn_auto-edge03
```

반면 Zabbix에서는 동일한 호스트를 다음과 같은 형식으로 관리하는 경우가 있었다.

```
auto-edge03_121.78.33.144
```

이러한 차이로 인해 다음 문제가 발생할 가능성이 있었다.

- 동일 호스트의 중복 등록

- 호스트 이관 과정에서의 감시 대상 누락
- 시스템별 알람 기준 불일치
- 장애 이력 및 성능 데이터 분산
- 템플릿과 스크립트의 중복 관리
- 운영자의 유지보수 공수 증가
- 단일한 모니터링 가시성 확보의 어려움

또한 기존 Zabbix 6.0 환경은 단일 VM 중심으로 운영되어 Server 또는 Database 장애가 전체 모니터링 서비스 중단으로 이어질 수 있었다. 따라서 단순한 버전 업그레이드가 아니라, 모니터링 플랫폼 전체를 고가용성 구조로 재설계할 필요가 있었다.

1.2. 프로젝트 목표

본 프로젝트의 목표는 다음과 같다.

- **모니터링 플랫폼 통합**
 - Nagios에서 관리하던 호스트와 서비스 검사 항목을 Zabbix로 이관하여 모니터링 시스템을 단일 플랫폼으로 통합한다.
- **Zabbix 플랫폼 고도화**
 - Zabbix 6.0 LTS를 Zabbix 7.0 LTS로 전환하고, 비동기식 데이터 수집과 Proxy Group 등 신규 기능을 적용한다.
- **데이터베이스 구조 개선**
 - 기존 MySQL 기반 Database를 PostgreSQL 16 + TimescaleDB 구조로 전환하여 대규모 시계열 데이터 처리와 장기 보관 효율을 높인다.
- **단일 장애 지점 제거**
 - Zabbix Server, Zabbix Proxy 및 PostgreSQL Database를 계층별로 이중화하여 단일 노드 장애가 전체 모니터링 중단으로 이어지는 구조를 개선한다.
- **성능 및 장애조치 검증**
 - 대규모 Dummy Host를 이용한 부하 테스트와 노드별 강제 장애 테스트를 통해 시스템의 처리 성능과 복구 동작을 검증한다.

2. Zabbix 7.0 업그레이드 및 아키텍처 전환

2.1. 주요 컴포넌트 변경

구분	기존 환경	개선 환경
모니터링 서버	Zabbix 6.0 LTS	Zabbix 7.0 LTS
Database	MySQL 8.0	PostgreSQL 16 + TimescaleDB
Agent	Zabbix Agent	Zabbix Agent 2
Server 구조	단일 노드	Active-Standby 이중화
Proxy 구조	단일 Proxy 또는 개별 할당	Proxy Group 이중화
Database HA	단일 Database	Patroni + etcd 기반 HA
Database 연결	Database 직접 연결	HAProxy 고정 엔드포인트
모니터링 플랫폼	Nagios + Zabbix 분산	Zabbix 단일 플랫폼

2.2. 비동기식 Poller 도입

기존 Zabbix 6.0의 동기식 수집 방식에서는 하나의 Poller 프로세스가 감시 대상에 요청을 전달한 뒤 응답이 반환될 때까지 해당 작업을 점유한다.

네트워크 응답이 느리거나 Timeout이 발생하는 대상이 많아질수록 Poller가 대기 상태로 유지되어 다음과 같은 문제가 발생할 수 있다.

- Poller Busy 증가
- 수집 Queue 누적
- Poller 프로세스 추가에 따른 메모리 사용 증가
- 응답 지연 대상에 의한 전체 수집 처리량 저하

Zabbix 7.0에서는 이벤트 루프 기반의 비동기 Poller를 지원하며, 하나의 프로세스가 다수의 검사를 동시에 처리할 수 있다.

이를 통해 기대할 수 있는 효과는 다음과 같다.

- 프로세스당 최대 약 1,000개의 동시 검사 처리
- I/O 대기 중 다른 검사 처리
- CPU 및 프로세스 운영 오버헤드 감소
- 전체 수집 처리량 향상
- 대규모 네트워크 및 서비스 감시 환경의 확장성 개선

2.3. Proxy Group 도입

기존 구조에서는 하나의 호스트가 특정 Proxy에 고정적으로 연결되므로, 해당 Proxy가 중단되면 다른 Proxy가 존재하더라도 자동으로 수집을 이어받지 못하는 구조적 한계가 있었다.

Zabbix 7.0의 Proxy Group은 여러 Proxy를 하나의 논리 그룹으로 구성하고, 그룹에 할당된 호스트를 Proxy 상태에 따라 분배한다.

주요 기능

- 로드밸런싱
 - Proxy Group Manager가 그룹 내 Proxy 상태를 확인하고 감시 대상을 분산한다.
- 자동 페일오버
 - 특정 Proxy가 설정된 Failover Period 동안 응답하지 않으면 해당 Proxy를 Offline으로 판단하고, 담당 호스트를 생존 Proxy로 재할당한다.
- 메모리 버퍼 활용
 - Proxy가 수집한 데이터를 Server로 즉시 전송하지 못하더라도 버퍼에 일정 기간 보관한 후 연결이 복구되면 다시 전송할 수 있다.
- 적용 효과
 - 단일 Proxy 장애 지점 제거
 - Proxy 유지보수 시 수집 서비스 영향 최소화
 - 감시 대상 증가 시 Proxy 수평 확장 가능
 - Active Agent 데이터의 일시적 연결 장애 대응
 - Proxy별 수집 부하 분산

2.4. Database 엔진 전환

기존 MySQL 기반 Database를 PostgreSQL 16 + TimescaleDB 구조로 전환하였다.

Zabbix는 시간 순서에 따라 지속적으로 History와 Trend 데이터를 저장하므로 일반적인 업무 Database와 비교해 대량의 시계열 쓰기 작업이 발생한다.

TimescaleDB는 PostgreSQL 확장 기능으로서 시계열 데이터를 시간 단위의 Chunk로 분할하여 관리한다.

주요 기대 효과

- 시계열 데이터 압축
 - 일정 기간이 지난 History 데이터를 압축하여 장기 보관에 필요한 저장 공간을 줄일 수 있다.
- 보관기간 만료 데이터 관리
 - 기존의 대량 **DELETE** 방식 대신 보관기간이 지난 Chunk를 제거하는 방식으로 데이터를 관리할 수 있다.
 - 대규모 삭제 작업에 따른 부하 감소
 - Dead Tuple 및 Vacuum 부담 완화
 - History 보관정책 관리 효율 향상
 - 장기간 데이터 축적 환경에서의 운영 안정성 개선
- 대량 데이터 쓰기 대응
 - PostgreSQL과 TimescaleDB를 이용하여 다수의 시계열 데이터가 지속적으로 입력되는 환경에 대응할 수 있는 구조를 마련하였다.

2.5. Zabbix Agent 2 적용

기존 Zabbix Agent 대신 Zabbix Agent 2를 적용하였다.

Zabbix Agent 2는 PostgreSQL, Redis, HAProxy, Docker 등 플러그인 기반 구조를 제공하므로 별도의 모니터링 스크립트를 다수 개발하지 않고도 다양한 애플리케이션을 감시할 수 있다.

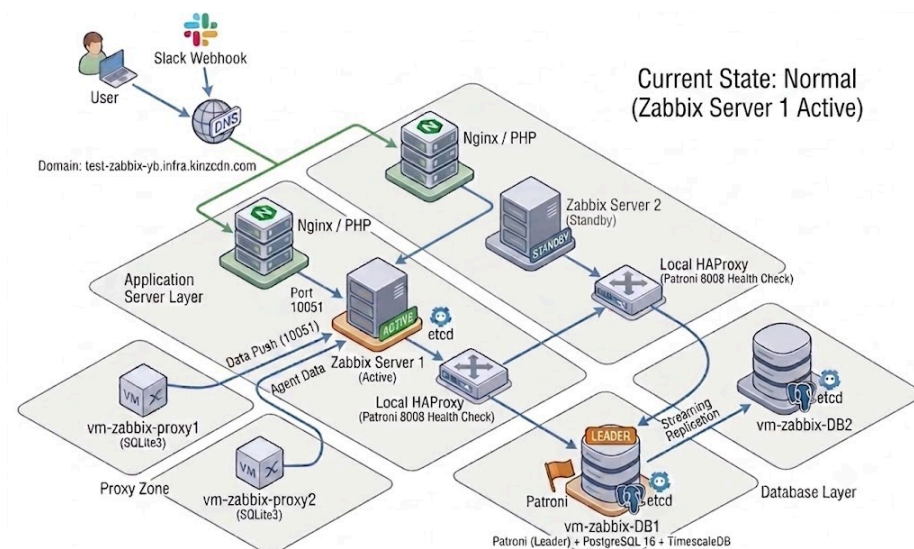
기대 효과

- 서드파티 스크립트 의존도 감소
- 애플리케이션별 표준 템플릿 적용
- 모니터링 항목의 일관성 확보
- 향후 서비스 추가 시 확장성 향상
- Agent 버퍼를 통한 일시적 통신 장애 대응

3.고가용성 Zabbix 아키텍처 구축

3.1. 전체 구성

본 프로젝트에서는 총 6개의 VM을 이용하여 Zabbix Server, Proxy 및 Database 계층을 각각 이중화하였다.



호스트명	사양	주요 역할
vm-zabbix-server1	4 Core / 8GB	Zabbix Server 1, Nginx/PHP, HAProxy, etcd Node 3
vm-zabbix-server2	4 Core / 8GB	Zabbix Server 2, Nginx/PHP, HAProxy
vm-zabbix-proxy1	2 Core / 4GB	Zabbix Proxy Group Node 1, SQLite3
vm-zabbix-proxy2	2 Core / 4GB	Zabbix Proxy Group Node 2, SQLite3
vm-zabbix-DB1	4 Core / 8GB	PostgreSQL 16, TimescaleDB, Patroni, etcd Node 1
vm-zabbix-DB2	4 Core / 8GB	PostgreSQL 16, TimescaleDB, Patroni, etcd Node 2

3.2. Patroni와 etcd 기반 Database HA

Patroni는 PostgreSQL 고가용성 구성을 관리하는 컴포넌트로, etcd를 분산 구성 저장소이자 리더 선출을 위한 합의 시스템으로 사용한다.

리더 선출

Patroni는 etcd에 저장된 분산 락과 Heartbeat 정보를 이용하여 하나의 PostgreSQL 노드만 Primary 역할을 수행하도록 관리한다.

Primary 노드가 정상적으로 동작하는 동안에는 주기적으로 리더 키의 TTL을 갱신한다.

Primary가 중단되어 TTL이 더 이상 갱신되지 않으면 Standby 노드가 리더 선출 절차에 참여하고, 조건을 만족하는 노드가 새로운 Primary로 승격된다.

Split-Brain 방지

각 Database 노드가 독립적으로 자신을 Primary로 판단하면 동일 데이터에 서로 다른 쓰기가 발생하는 Split-Brain 문제가 발생할 수 있다.

이를 방지하기 위해 3개의 etcd 노드를 구성하고 과반수 합의를 기반으로 리더를 결정하였다.

- etcd Node 1: vm-zabbix-DB1
- etcd Node 2: vm-zabbix-DB2
- etcd Node 3: vm-zabbix-server1

자동 복제 구성

Replica Database가 처음 구성되거나 재구축될 때 Patroni가 Primary 데이터를 기준으로 복제 관계를 구성한다.

PostgreSQL 프로세스 관리

Patroni가 PostgreSQL 프로세스를 소유하고 제어하므로 PostgreSQL 서비스의 설정들을 모든 DB노드에 일관되게 적용할 수 있다.

3.3. Patroni REST API

Patroni는 각 노드에서 REST API를 제공한다.

Primary 확인을 위해 다음 Endpoint를 사용할 수 있다.

```
GET /primary
```

응답 예시는 다음과 같다.

노드 상태	응답
현재 Primary	HTTP 200
Standby 또는 비정상 노드	HTTP 503

3.4. HAProxy 기반 Database 라우팅

각 Zabbix Server에는 로컬 HAProxy를 구성하였다.

Zabbix Server는 실제 Database 노드의 주소를 직접 바라보지 않고 다음과 같이 로컬 고정 Endpoint에 연결한다.

```
127.0.0.1:5432
```

HAProxy는 Patroni의 `/primary` API를 호출하여 현재 Primary Database를 확인하고, Zabbix Server의 Database 연결을 해당 노드로 전달한다.

주요 역할

- 고정 Endpoint 제공
 - Primary Database가 DB1에서 DB2로 변경되더라도 Zabbix Server의 Database 접속 설정은 변경하지 않는다.
- Primary 상태 확인
 - PostgreSQL TCP Port만 확인하지 않고 Patroni REST API의 응답 상태를 기준으로 Backend를 선택한다.
- 기존 세션 정리
 - 기존 Primary가 중단되거나 Standby로 강등될 경우 연결된 Database 세션을 종료하여 Zabbix Server가 연결 단절을 빠르게 인지하고 새로운 Primary로 재접속하도록 한다.

3.5. Zabbix Server HA

Zabbix Server 계층은 Zabbix 자체 HA 기능을 이용하여 Active-Standby 형태로 구성하였다.

두 Server 노드는 동일한 Database에 자신의 상태를 주기적으로 기록한다.

정상 동작

- Active Server가 데이터 수집 관리, Trigger 평가 및 알림 발송 수행
- Standby Server는 Active Server의 상태를 확인하며 대기
- 두 노드가 동시에 중앙 처리 기능을 수행하지 않도록 제어

장애조치

Active Server의 Heartbeat 갱신이 중단되면 Standby Server가 이를 감지하고 Active로 승격된다.

Zabbix는 수집 데이터 처리와 Trigger 평가, 이벤트 생성 및 Database 쓰기 작업을 중앙에서 처리하므로 Active-Active가 아닌 Active-Standby 구조를 적용하였다.

4. 대규모 부하 테스트 및 최적화

4.1. 테스트 목적

고가용성 아키텍처가 단순히 서비스 기동에 성공하는지를 확인하는 것만으로는 실제 운영 가능성을 판단하기 어렵다.

따라서 다음 항목을 검증하기 위해 대규모 부하 테스트를 수행하였다.

- 5,000 NVPS 환경에서의 데이터 수집 안정성
- Zabbix Server 및 Proxy의 메모리 사용 상태
- Database 처리 부하
- 수집 Queue 누적 여부
- 고부하 상태에서의 장애조치 동작
- 설정 변경이 전체 시스템에 미치는 영향

4.2. 테스트 부하 산정

Zabbix 환경의 처리 규모는 일반적으로 NVPS, 즉 초당 새로운 값의 수로 표현할 수 있다.

```
NVPS = 전체 수집 아이템 수 ÷ 평균 수집 주기
```

본 프로젝트에서는 목표 수치를 약 5,000 NVPS로 설정하였다.

4.3. 테스트 환경 구성

Python Multithreading과 `zabbix_sender`를 이용해 부하 발생기를 구현하였다.

Dummy Host 구성

- 부하 발생 인스턴스: 4대
- 인스턴스당 Dummy Host: 500개
- 전체 Dummy Host: 2,000개

Dummy Template 구성

- 호스트당 Item: 25개
- 호스트당 Trigger: 25개
- 데이터 전송 주기: 10초
- Item Type: Trapper 기반

NVPS 계산

```
2,000 Hosts × 25 Items = 50,000 Values  
50,000 Values ÷ 10 Seconds = 5,000 NVPS
```

이를 통해 10초마다 약 50,000개의 데이터를 전송하는 5,000 NVPS 환경을 구현하였다.

4.4. Proxy 공유 메모리 고갈

부하 테스트를 시작한 직후 Zabbix Proxy 프로세스가 공유 메모리 부족으로 비정상 종료되는 문제가 발생하였다.

오류 메시지

```
[file:dbconfig.c,line:233] __zbx_shmem_realloc(): out of memory  
(requested 24 bytes)  
  
[file:dbconfig.c,line:233] __zbx_shmem_realloc():  
please increase CacheSize configuration parameter
```

원인

Zabbix Proxy가 Server로부터 전달받은 다수의 호스트, 아이템 및 Trigger 설정 정보를 공유 메모리에 적재하는 과정에서 기본 CacheSize를 초과하였다.

테스트 환경에서는 2,000개 호스트에 각각 다수의 아이템과 Trigger가 연결되어 Proxy가 처리해야 하는 메타데이터 규모가 크게 증가하였다.

조치

`/etc/zabbix/zabbix_proxy.conf`에서 다음과 같이 CacheSize를 변경하였다.

```
CacheSize=256M
```

설정 변경 후 Proxy를 재시작하였다.

```
sudo systemctlrestart zabbix-proxy
```

결과

공유 메모리 부족으로 인한 Proxy 프로세스 비정상 종료 현상이 해소되었으며, 이후 5,000 NVPS 부하를 지속적으로 처리할 수 있었다.

4.5. 6시간 부하 테스트 결과

CacheSize 조정 이후 약 6시간 동안 5,000 NVPS 부하 테스트를 수행하였다.

확인 결과

- Zabbix Server 정상 동작
- Zabbix Proxy 데이터 수집 유지
- Proxy 메모리 Crash 재발 없음
- Trigger 및 Item 데이터 입력 유지
- Database Utilization 약 60~80%
- 시간당 로그 발생량 약 3GB

분석

Server와 Proxy 계층에서는 테스트를 중단시킬 수준의 추가 장애가 발생하지 않았다.

다만 Database Utilization이 지속해서 60~80% 수준을 유지했으므로 실제 운영 환경에서 다음 조건이 증가하면 Database 병목이 발생할 가능성이 있다.

- NVPS 증가
- History 보관기간 증가
- Trigger Expression 증가
- 동시 사용자 및 Dashboard 조회 증가
- Housekeeping 및 압축 작업 동시 수행
- Trend 및 Report 조회 증가

4.6. 운영 권장사항

Database 사양 확장

현재 테스트 Database 사양은 4 Core / 8GB이다.

운영 NVPS가 테스트 수준에 근접하거나 이를 초과할 경우 다음과 같은 확장을 검토한다.

```
8 Core / 32GB 이상
```

5. 장애조치 시나리오 검증

5.1. 검증 방법

5,000 NVPS 부하 테스트를 유지한 상태에서 각 계층의 VM을 일시중지하여 장애조치 동작을 검증하였다.

검증 대상은 다음과 같다.

1. Primary PostgreSQL Database
2. Active Zabbix Server
3. Zabbix Proxy Group의 Proxy 1

측정 기준

- 장애 발생 시점
- Standby 승격 시점
- HAProxy 또는 Zabbix Server의 장애 인식 시점
- 정상 데이터 수집 재개 시점
- 전체 서비스 복구 시간

5.2. 검증 한계

본 테스트는 주로 시스템로그를 기반으로 분석하였다.

장애 전후 Database에 저장된 전체 History Record 수와 Timestamp를 별도로 비교하지 않았으므로 데이터 유실 범위와 정확한 RPO를 확정할 수 없다.

5.3. 시나리오 1: Primary Database 장애

테스트 방법

현재 Primary 역할을 수행 중인 Database VM을 일시중지하였다.

동작 과정

1. Primary DB Heartbeat 중단
2. Patroni와 etcd가 기존 Primary 장애 감지
3. Standby DB가 새로운 Primary로 승격
4. 새로운 Primary의 `/primary` API가 HTTP 200 반환
5. HAProxy가 Backend를 새로운 Primary로 전환
6. Zabbix Server가 Database에 재접속
7. 수집 및 Database 쓰기 재개

측정 결과

항목	결과
Standby 승격 및 HAProxy Primary 인식	약 14.6초
Zabbix Server Database 재연결 및 완전 복구	약 18.6초

데이터 영향 분석

Zabbix Server는 수집 데이터를 Database에 기록하기 전에 History Cache에 임시 보관한다.

Database 장애 복구 시간이 1분 이내였으며 Cache가 고갈되기 전에 연결이 복구되었으므로, History Cache에 있던 데이터가 복구 후 Database로 기록되었을 가능성이 높다.

그러나 장애 전후 Database Record를 직접 비교하지 않았으므로 다음과 같이 평가한다.

테스트 로그와 Cache 동작을 기준으로 관측상 데이터 유실은 확인되지 않았으나, 정확한 RPO는 산정하지 못하였다.

5.4. 시나리오 2: Active Zabbix Server 장애

테스트 방법

현재 Active 역할을 수행 중인 Zabbix Server VM을 일시중지하였다.

동작 과정

1. Active Server Heartbeat 갱신 중단
2. Standby Server가 Active Server 장애 감지
3. Standby Server가 Active로 승격
4. Proxy Group과 연결
5. 데이터 수집 및 Trigger 평가 재개

측정 결과

항목	결과
Standby Server의 Active 승격	약 2.55초
Proxy Group 연결을 포함한 완전 복구	약 6.6초

데이터 영향 분석

Proxy가 수집하여 아직 Server로 전송하지 못한 데이터는 Proxy 버퍼에 저장되므로, Proxy 측 수집 데이터는 Server 복구 후 재 전송될 수 있다.

다만 기존 Active Server가 Proxy로부터 데이터를 전달받은 후 Database에 Flush하기 전에 VM이 중단된 경우, Server 메모리에만 존재하던 일부 데이터가 유실될 가능성이 있다.

따라서 다음과 같이 평가한다.

Zabbix Server HA 전환은 약 6.6초 내에 완료되었으며 Proxy 버퍼 데이터는 보존되었다. 다만 장애 순간 Active Server의 메모리에만 존재하던 일부 데이터는 유실되었을 가능성이 있다.

5.5. 시나리오 3: Zabbix Proxy 장애

테스트 방법

Proxy Group에서 동작 중인 Proxy 1 VM을 일시중지 하였다.

동작 과정

1. Proxy 1 응답 중단
2. 설정된 Failover Period 동안 장애 여부 확인
3. Server가 Proxy 1을 Offline으로 판단
4. Proxy 1 담당 호스트를 Proxy 2로 재할당
5. Proxy 2로 호스트 및 Item 메타데이터 전송
6. Proxy 2가 재할당된 호스트 수집 시작
7. Active Agent의 Buffer 데이터 추가 유입

측정 결과

항목	결과
Proxy Offline 판정	약 63초
호스트 재할당 및 설정 전송	약 7초

항목	결과
전달된 설정 데이터	약 3MB
전체 Proxy 전환 시간	약 70.7초
이후 Agent Buffer 데이터 유입	약 13초

수집 방식별 영향

Active 수집

Active Agent 또는 Trapper 방식은 감시 대상이 직접 Proxy로 데이터를 전송한다.

기존 Proxy에 연결할 수 없는 동안 Agent가 데이터를 Buffer에 저장하고, 새로운 Proxy 연결 후 전송하므로 관측상 데이터 공백을 줄일 수 있다.

Passive 수집

SNMP, ICMP Ping, Zabbix Passive Agent 및 Simple Check 등은 Proxy가 대상에 직접 요청을 보내야 한다.

Proxy가 중단된 후 다른 Proxy가 호스트를 인계받기 전까지 Polling을 수행할 주체가 없으므로 약 70초 동안 데이터 공백이 발생하였다.

5.6. 장애조치 결과 요약

장애 대상	전체 복구 시간	데이터 영향	평가
Primary Database	약 18.6초	관측상 유실 없음, 정확한 RPO 미산정	자동 승격 및 HAProxy 전환 확인
Active Zabbix Server	약 6.6초	종료 직전 1~3초 데이터 유실 가능성	Standby 자동 승격 확인
Zabbix Proxy	약 70.7초	Active 수집은 Buffer 복구, Passive 수집은 공백 발생	Proxy Group 재할당 확인

종합 평가

- Database, Server 및 Proxy 계층의 자동 장애조치가 정상적으로 동작하였다.
- Server 장애조치는 수 초 수준으로 수행되었다.
- Database 장애조치는 수십 초 이내에 완료되었다.
- Proxy 장애조치는 기본 Failover Period의 영향을 크게 받았다.
- Active 수집은 Agent Buffer를 통해 장애 구간 데이터 보존에 유리했다.
- Passive 수집은 Proxy 전환 시간 동안 데이터 공백이 발생했다.
- 정확한 RPO 측정을 위해 Database Record 기반 추가 검증이 필요하다.

6. Nagios와 Zabbix 모니터링 플랫폼 통합

6.1. 호스트 전수 매핑

Nagios에는 총 259개의 호스트가 등록되어 있었으며, Zabbix의 Hostname과 Naming Convention이 일치하지 않는 항목이 다수 존재하였다.

이에 따라 IP 주소를 기준으로 Nagios와 Zabbix 호스트를 교차 검증하였다.

처리 방식

- **중복 호스트**
 - Nagios와 Zabbix에서 동일한 IP를 감시하는 경우 Zabbix의 단일 호스트 객체를 기준으로 통합하였다.
- **Nagios 단독 호스트**

- Zabbix에는 존재하지 않고 Nagios에서만 감시하던 약 20개의 호스트는 기존 Zabbix Naming Convention에 맞춰 신규 등록하였다.
- 불명확한 호스트
 - IP 주소 또는 Hostname만으로 동일 대상을 확정하기 어려운 경우 운영 담당자의 추가 확인이 필요한 항목으로 분류하였다.

Nagios와 Zabbix의 Hostname 차이로 인한 중복·누락을 방지하기 위해 IP와 Hostname을 함께 대조하였으며, 중복 대상은 병합하고 Nagios 단독 대상은 신규 등록하였다.

6.2. 서비스 검사 항목 이관

Nagios에서 사용하던 주요 검사 항목을 분석하고 Zabbix 기본 기능 또는 커스텀 템플릿으로 이관하였다.

Nagios 검사 항목	검사 목적	Zabbix 이관 방식
HTTP OK	응답 코드, 응답 속도 및 페이지 상태 확인	HTTP Agent
SSL Certificate	인증서 유효성 및 만료일 확인	Website Certificate by Zabbix Agent 2
Ping Check	ICMP 응답, Packet Loss 및 RTT 확인	ICMP Ping Template
DNS Check	지정 DNS 서버 응답과 지연 시간 확인	<code>dig</code> 기반 커스텀 DNS Template
RTMP Check	RTMP 스트리밍 서비스 정상 여부 확인	<code>rtmpdump</code> 기반 External Check
Docker Container	Container 상태 및 자원 사용량	Zabbix Agent 2 Docker Plugin

HTTP, SSL 및 Ping은 Zabbix 기본 기능 또는 템플릿으로 대체할 수 있었지만, 기존 운영 방식과 동일한 DNS 및 RTMP 검사는 별도의 커스텀 구현이 필요했다.

7. DNS Check 커스텀 템플릿 개발

7.1. 기존 검사 방식

Nagios에서는 지정한 Domain을 각각 다른 DNS Server에 질의하고 다음 항목을 확인하고 있었다.

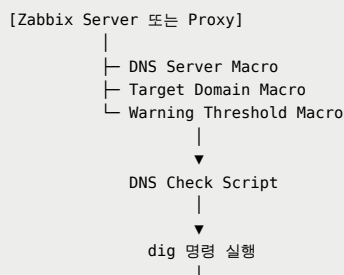
- DNS 질의 성공 여부
- 응답 지연 시간
- 반환된 IP 주소
- Timeout 여부

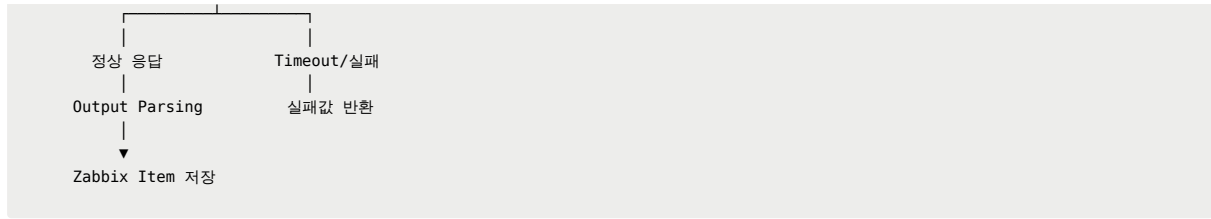
Zabbix 기본 DNS 관련 기능만으로는 기존 검사 방식의 모든 인자를 동일하게 템플릿화하기 어려워 커스텀 스크립트를 개발하였다.

7.2. 구현 방식

Zabbix Macro를 이용하여 검사 대상 정보를 스크립트에 전달하도록 구성하였다.

처리 절차





주요 동작

1. Zabbix가 Macro 값을 스크립트에 전달
2. 스크립트에서 `dig` 명령 실행
3. 지정 DNS Server에 대상 Domain 질의
4. 최대 10초를 초과하면 실패 처리
5. `dig` 출력값에서 응답 상태 및 지연 시간 추출
6. 결과를 Zabbix에 반환
7. Item 및 Trigger를 이용하여 상태 감시

7.3. 적용 결과

개발한 DNS Check Template을 실제 호스트에 적용하여 다음을 확인하였다.

- Macro 값 정상 전달
- 지정 DNS Server 질의 성공
- 응답값 정상 파싱
- Zabbix Latest Data 입력
- Timeout 및 실패 상태 구분
- 호스트별 재사용 가능

DNS Server와 Domain 정보를 Macro로 분리함으로써 대상이 추가될 때마다 스크립트를 새로 생성하지 않고 동일한 템플릿을 재 사용할 수 있도록 개선하였다. DNS 검사 스크립트는 `dig` 결과를 파싱하고 10초 초과 시 실패로 처리하도록 구현하였다.

8. RTMP Stream Check 템플릿 개발

8.1. 기존 환경의 문제점

기존 RTMP 스트리밍 검사는 웹 서버 디렉터리에 호스트별 PHP 파일을 생성하는 방식으로 운영되었다.

예시는 다음과 같다.

```

wz01.php
wz02.php
wz201.php
cdnlxwow.php
...
  
```

각 PHP 파일은 RTMP 대상 IP만 다르고 내부적으로 `rtmpdump`를 실행하는 로직은 대부분 동일했다.

따라서 새로운 RTMP 서버를 추가할 때마다 다음 작업이 필요했다.

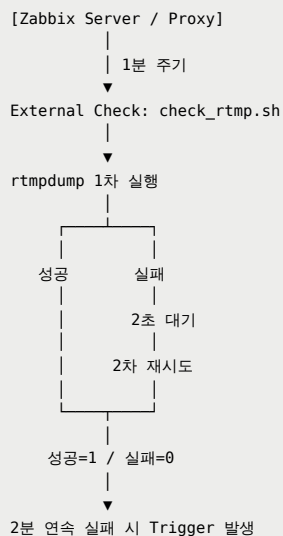
1. 기존 PHP 파일 복제
2. 대상 IP 수정
3. 웹 서버에 파일 배포

4. 파일 권한 설정
5. Nagios 또는 웹 검사 항목 추가
6. 장애 기준 개별 설정

이 구조는 대상 수가 증가할수록 관리 파일과 설정이 함께 증가하는 문제가 있었다.

8.2. 개선 구조

호스트별 PHP 파일을 제거하고 Zabbix External Check 기반의 공통 스크립트와 템플릿을 구성하였다.



8.3. Item 구성

설정 항목	값
Template	RTMP Stream Monitoring
Type	External Check
Script	check_rtmp.sh
Type of Information	Numeric unsigned
정상값	1
실패값	0
Update Interval	1m

Item Key

```
check_rtmp.sh["rtmp://{HOST.CONN}/monitoring/mp4:sample_check.mp4"]
```

{HOST.CONN} Macro를 이용하여 현재 호스트의 연결 IP를 RTMP URL에 전달한다.

이에 따라 대상이 추가되더라도 스크립트나 PHP 파일을 새로 만들 필요 없이 다음 작업만 수행하면 된다.

1. Zabbix Host 등록
2. Host Interface IP 설정
3. RTMP Stream Monitoring Template 연결

8.4. 스크립트 동작

스크립트는 `rtmpdump` 를 이용하여 지정 RTMP 주소로 스트리밍 요청을 수행한다.

성공 조건

- RTMP 연결 성공
- 스트림 요청 성공
- 결과 파일 생성 또는 정상 응답 확인

실패 조건

- 연결 Timeout
- 대상 Port 접근 실패
- Stream Path 오류
- `rtmpdump` 비정상 종료
- 파일 생성 실패

반환값

정상: 1
장애: 0

8.5. 오탐 방지

기존 운영환경에서 Zabbix 전환 시 오탐 증가 가능성이 제기되었으므로 Script와 Trigger 두 단계에 재확인 로직을 적용하였다.

Script 재시도

첫 번째 검사 실패 시 즉시 장애로 판단하지 않고 2초 후 한 차례 재시도한다.

1차 성공 → 1 반환
1차 실패 → 2초 대기
2차 성공 → 1 반환
2차 실패 → 0 반환

Trigger 지속 조건

단일 실패값만으로 장애 이벤트를 발생시키지 않고 최근 2분 동안 실패 상태가 지속되는 경우에만 Trigger를 발생시킨다.

```
max(
  /RTMP Stream Monitoring/check_rtmp.sh[
    "rtmp://{HOST.CONN}/monitoring/mp4:sample_check.mp4"
  ],
  2m
)=0
```

이 조건은 최근 2분 동안 수집된 값의 최댓값이 0인 경우, 즉 해당 기간에 한 번도 성공값 1이 들어오지 않은 경우에만 장애로 판단한다. 최종 PPT에서도 RTMP Trigger 조건을 최근 2분간의 연속 실패 기준으로 구성하였다.

8.6. 개선 효과

구분	기존 방식	개선 방식
관리 단위	호스트별 PHP 파일	공통 Script 1개
대상 IP	PHP 파일 내부에 직접 입력	<code>{HOST.CONN}</code> Macro
대상 추가	파일 생성 및 검사 등록	Host 등록 및 Template 연결
오탐 방지	개별 구현	Script 재시도 + Trigger 지속 조건

구분	기존 방식	개선 방식
결과 형식	HTTP/PHP 응답	Numeric 1 또는 0
유지보수	파일 수에 비례하여 증가	템플릿 중심 중앙 관리

9. 기존 Zabbix 6.0 템플릿 이관

9.1. 이관 원칙

기존 Zabbix 6.0에서 사용하던 커스텀 템플릿은 운영 중 검증된 항목과 임계치를 포함하고 있으므로 일괄 삭제하거나 Zabbix 7.0 기본 템플릿으로 즉시 교체하지 않았다.

다음 원칙을 적용하였다.

- 기존 운영 항목 최대한 보존
- Zabbix 7.0 기본 Template 우선 적용
- 중복 Item 제거
- 호환되지 않는 Item 분리
- 회사별 Custom Item은 Extension Template으로 관리
- 운영 전 병행 수집을 통해 결과 비교

9.2. Linux 템플릿 정리

기존 Linux Custom Template에는 Zabbix 7.0의 `Linux by Zabbix agent` 와 중복되거나 충돌할 수 있는 Item이 존재했다.

이에 따라 다음과 같이 분리하였다.

기본 항목

Linux by Zabbix agent

기존 환경에만 존재하는 추가 항목

Linux Custom Extension

이를 통해 Zabbix 공식 템플릿의 개선사항을 적용하면서도 기존 환경에서 별도로 사용하던 Item을 유지할 수 있도록 하였다.

9.3. Windows 템플릿 정리

Windows 환경도 동일한 기준을 적용하였다.

- Zabbix 7.0 기본 Windows Template 적용
- 기존 Custom Item 중 중복 항목 제외
- 회사 운영환경에 필요한 항목 별도 Extension 구성
- Trigger Name 및 Severity 정규화
- 기존 알람 기준과 신규 템플릿 기준 비교

기존 커스텀 템플릿을 그대로 폐기하지 않고 기본 템플릿과 Extension 템플릿으로 분리하여 버전 호환성과 운영 연속성을 함께 확보하였다.

10. 주요 트러블슈팅 사례

10.1. Zabbix Proxy 공유 메모리 부족

증상

5,000 NVPS 부하 테스트 시작 직후 Proxy가 공유 메모리 할당에 실패하며 비정상 종료되었다.

```
__zbx_shmem_realloc(): out of memory  
please increase CacheSize configuration parameter
```

원인

2,000개 호스트에 연결된 Item과 Trigger 메타데이터를 Proxy가 메모리에 적재하면서 기본 CacheSize를 초과하였다.

조치

```
CacheSize=256M
```

설정 변경 후 Proxy를 재시작하였다.

결과

약 6시간 동안 5,000 NVPS 부하를 안정적으로 처리했으며 동일한 공유 메모리 오류는 재발하지 않았다.

교훈

대규모 환경에서는 단순 NVPS뿐 아니라 Proxy가 관리하는 호스트, Item, Trigger 및 Discovery Rule 수를 기준으로 Config Cache 용량을 산정해야 한다.

10.2. Patroni 클러스터 Bootstrap 실패

증상

PostgreSQL 노드 두 대를 Patroni Cluster로 구성한 후 두 노드 모두 Replica 상태에 머물렀으며 Primary가 정상적으로 선출되지 않았다.

원인

기존 PostgreSQL 설치 및 초기 설정 과정에서 Database Data Directory에 파일이 생성되어 있었다.

완전히 비어 있지 않은 Data Directory와 기존 PostgreSQL 상태 정보로 인해 Patroni가 신규 Cluster Bootstrap을 정상적으로 수행하지 못했다.

조치

1. PostgreSQL 서비스 중지
2. Patroni 서비스 중지
3. 기존 Data Directory 제거
4. Directory 재생성
5. 소유권과 권한 재설정
6. Patroni 설정 검토
7. Patroni 재기동
8. Primary 선출 확인
9. Replica의 `pg_basebackup` 및 복제 상태 확인

결과

한 노드가 Primary로 정상 선출되고 다른 노드는 Replica로 구성되었으며 Streaming Replication이 정상화되었다.

교훈

Patroni가 PostgreSQL 프로세스를 관리하는 구조에서는 기존 PostgreSQL 서비스와 Data Directory 상태를 명확하게 정리한 뒤 Bootstrap을 진행해야 한다.

10.3. Zabbix Proxy와 HA Server 간 연동 실패

증상

Proxy 설정을 변경하였으나 이중화된 Zabbix Server와 정상적으로 통신하지 못했으며, Proxy가 수집한 데이터가 Server로 전달되지 않았다.

원인

Zabbix Proxy 또는 Agent 설정에서 복수 Server 주소를 지정할 때 사용되는 구분자와 연결 동작을 정확히 반영하지 못했다.

설정값의 주소 표기 방식에 따라 여러 Server를 동일 그룹으로 허용하거나 우선순위를 두고 연결하는 동작이 달라질 수 있으므로, Proxy Mode와 Server 측 허용 설정을 함께 검토해야 했다.

조치

- Proxy의 Active·Passive Mode 확인
- **Server** 및 관련 Server 주소 설정 검토
- 복수 Server 주소 표기 방식 수정
- Server의 Proxy 등록 Hostname 확인
- 방화벽 및 Port 확인
- Proxy Log에서 연결 대상 확인
- Active Server 승격 후 재연결 확인

결과

설정 수정 후 Proxy가 이중화된 Zabbix Server와 정상적으로 통신하고 수집 데이터를 전달하였다.

교훈

HA 환경에서는 단순히 복수 IP를 나열하는 것뿐 아니라 다음 항목을 함께 검증해야 한다.

- Proxy 동작 모드
- Server와 Proxy의 연결 방향
- Hostname 일치 여부
- 허용 Server 목록
- 주소 구분자 해석
- Active Server 변경 시 재연결 동작

11. 단계적 마이그레이션 정책

11.1. 기본 원칙

모니터링 시스템을 장애를 감지하고 운영자에게 전달하는 핵심 인프라이므로 신규 시스템 구축이 완료되었다고 해서 기존 시스템을 즉시 종료하는 것은 위험하다.

따라서 신규 Zabbix 7.0과 기존 Nagios 및 Zabbix 6.0을 일정 기간 병행 운영하는 단계적 전환 방식을 적용한다.

11.2. 1단계: 병행 운영 및 검증

권장 기간

구축 완료 후 약 1개월

주요 작업

- 신규·기존 시스템 동시 수집
- Trigger 발생 여부 비교
- 알람 누락 확인
- 오탐 및 과도한 알람 조정
- 담당자별 Dashboard 및 알람 채널 검증

11.3. 2단계: 기존 시스템 조회 전용 운영

신규 시스템이 안정화된 이후 기존 Zabbix 6.0은 실시간 모니터링 역할을 중단하고 과거 데이터 조회용으로 전환한다.

주요 작업

- 기존 `zabbix-server` 프로세스 중지
- Nginx 또는 Apache와 Database만 필요 시 구동
- 기존 알람 발송 차단
- 읽기 전용 성격으로 운영
- VM Snapshot 및 Database Backup 유지

이를 통해 중복 알람을 방지하면서 과거 Trend 및 장애 이력을 일정 기간 조회할 수 있다.

11.4. 3단계: 폐기 또는 아카이빙

권장 시점

신규 시스템 전환 후 약 1년

처리 방식

- 법적·운영상 보관기간 확인
- Database Dump 생성
- VM Snapshot 생성
- 오프라인 Storage로 이전
- 복구 절차 문서화
- 운영 VM 종료 및 자원 반납

최종적으로 기존 모니터링 시스템의 컴퓨팅 자원을 회수하여 불필요한 운영비용을 줄인다.

12. 향후 발전 과제

12.1. Multi-Zone 아키텍처

현재 구축 환경은 단일 가용 영역 내에 배치되어 있으므로 VM 단위 장애에는 대응할 수 있지만 가용 영역 또는 데이터센터 전체 장애에는 취약하다.

향후 권한 및 인프라 조건이 확보되면 다음과 같은 Multi-Zone 배치를 검토한다.

Zone A

- Zabbix Server 1
- Zabbix Proxy 1
- PostgreSQL DB1
- etcd Node 1

Zone B

- Zabbix Server 2
- Zabbix Proxy 2
- PostgreSQL DB2
- etcd Node 2

외부 또는 제3 Zone

- etcd Node 3
- 백업 또는 DR 구성요소

12.6. 모니터링 호스트와 템플릿 최신화

기존에 장기간 사용한 호스트와 템플릿에는 다음 항목이 남아 있을 수 있다.

- 더 이상 사용하지 않는 Host
- 중복 Item
- Unsupported Item
- 과도하게 짧은 수집 주기
- 오래된 UserParameter
- 사용되지 않는 Trigger
- 담당자 없는 알림
- 기준이 불명확한 Severity
- 오래된 Agent Version
- 잘못된 Interface IP
- 해제되지 않은 Maintenance

정기적인 Host 및 Template 정비를 통해 수집 부하와 운영 복잡도를 줄여야 한다.

13. 프로젝트 결과

13.1. 정량적 결과

항목	결과
Nagios 분석 대상	259개 호스트
Nagios 단독 신규 등록 대상	약 20개
HA 구성 노드	총 6대
부하 테스트 Dummy Host	2,000개
호스트당 Item	25개

항목	결과
호스트당 Trigger	25개
데이터 전송 주기	10초
테스트 부하	약 5,000 NVPS
부하 테스트 시간	약 6시간
DB Utilization	약 60~80%
로그 발생량	시간당 약 3GB
Database 장애 복구	약 18.6초
Zabbix Server 장애 복구	약 6.6초
Proxy 장애 복구	약 70.7초

13.2. 정성적 결과

- 모니터링 플랫폼 단일화 기반 확보

- Nagios와 Zabbix로 분산된 호스트 및 서비스 검사 항목을 Zabbix 7.0으로 통합할 수 있는 구조를 마련하였다.

- 단일 장애 지점 감소

- Server, Proxy 및 Database 계층을 이중화하고 자동 장애조치 동작을 확인하였다.

- 대규모 수집 환경 검증

- 2,000개 Dummy Host와 5,000 NVPS 부하를 이용하여 초기 성능 병목을 식별하고 Cache 설정을 최적화하였다.

- 커스텀 검사 항목 표준화

- DNS와 RTMP 검사를 개별 파일 중심 구조에서 Macro와 Template 중심 구조로 전환하였다.

- 운영 전환 정책 수립

- 즉시 종료 방식이 아닌 병행 운영, 조회 전용 전환, 아카이빙의 단계적 마이그레이션 정책을 수립하였다.

- 트러블슈팅 자산 확보

- 구축 과정에서 발생한 Proxy OOM, Patroni Bootstrap 및 HA Server 연동 문제의 원인과 해결 절차를 문서화하였다.

14. 결론

본 프로젝트에서는 기존 Nagios와 Zabbix 6.0 LTS로 파편화되어 있던 모니터링 환경을 Zabbix 7.0 LTS 기반의 통합 플랫폼으로 전환하기 위한 구조를 설계하고 구현하였다.

Database를 PostgreSQL 16 + TimescaleDB로 전환했으며, Patroni, etcd 및 HAProxy를 결합하여 Database 자동 장애조치 구조를 구성하였다. 또한 Zabbix Server의 Active-Standby HA와 Proxy Group을 적용하여 Application, 수집 및 Database 계층의 단일 장애 지점을 줄였다.

Nagios에 등록된 259개 호스트를 IP와 Hostname 기준으로 전수 검토하여 중복 호스트를 통합하고 Nagios에만 존재하던 약 20개 호스트를 Zabbix에 신규 등록하였다. HTTP, SSL 및 Ping 검사는 Zabbix 기본 기능으로 이관했으며, 기본 기능만으로 기존 요구사항을 충족하기 어려웠던 DNS와 RTMP 검사는 별도의 Script와 Template으로 개발하였다.

성능 검증에서는 2,000개 Dummy Host를 구성하여 약 5,000 NVPS의 부하를 구현하였다. 초기에는 Zabbix Proxy의 공유 메모리 부족 문제가 발생했으나 `CacheSize=256M`으로 조정하여 문제를 해결했으며, 이후 약 6시간 동안 부하를 유지하였다.

장애조치 테스트에서는 Database 약 18.6초, Zabbix Server 약 6.6초, Proxy 약 70.7초의 복구 시간을 확인하였다. 이를 통해 각 계층의 자동 장애조치가 정상적으로 동작하는 것을 확인하였다.

본 프로젝트를 통해 단일 플랫폼 기반의 모니터링 체계, 대규모 데이터 수집에 대응할 수 있는 확장 구조, 장애 발생 시 자동 복구가 가능한 고가용성 아키텍처를 확보하였다. 향후에는 Multi-Zone 배치, 제3 etcd 중재 노드 분리, Database 사양 확장, 기존 템플릿 최신화 및 RPO 정밀 검증을 수행함으로써 데이터센터 단위 장애에도 대응할 수 있는 모니터링 플랫폼으로 발전시킬 수 있다.